

Clickbait Spoiler Detection & Generation

<https://github.com/l1lunalu/clickbait-detection>

Luna Lu

University of Waterloo
Waterloo, Ontario, Canada
l351lu@uwaterloo.ca

Yibei Zheng

University of Waterloo
Waterloo, Ontario, Canada
y354zhen@uwaterloo.ca

Abstract

Clickbait content often uses hidden key information to intrigue users, therefore becoming a challenge in spoiler detection & generation and a valuable task in the field of natural language processing. This project addresses this problem by proposing two sub-tasks: (1) spoiler type classification, where the goal is to categorize the expected spoiler into one of three types—"phrase", "passage", or "multi"; and (2) spoiler generation, which aims to generate informative spoiler texts that reveal the actual content of the linked article.

For task 1, we experimented with various models ranging from a basic TF-IDF model to transformer-based architectures. The final model is built on "microsoft / deberta-v3 base" with an added custom classification head, which achieves a macro F1 score of 0.7317, significantly outperforming all previous models and the naive baseline model (F1 score 0.2085).

For Task 2, we explored several strategies, including keyword matching, type-guided generation, and zero-shot summarization. We ultimately fine-tuned a T5-large model in a sequence-to-sequence framework, which combined clickbait text, article title, and body paragraphs. This approach achieved a METEOR score of 0.365, significantly higher than the naive baseline model - simply use the clickbait spoilers as the real headlines (0.0924).

The results demonstrate that pre-trained language models exhibit strong capabilities for both classification and generation tasks in a multi-source setting. This project also highlights the importance of model selection, architecture depth, and careful input formatting for downstream performance. Future research may explore joint modeling of spoiler types and generation, as well as controllable decoding based on spoiler types.

1 Introduction

Clickbait is nowadays a common phenomenon in online media, characterized by deliberately designing eye-catching headlines to arouse users' curiosity without revealing the main content. Although such practice may increase user engagement, it often reduces the quality of information dissemination and aggravates user frustration. Therefore, automatically detecting and neutralizing clickbait has become a meaningful task in natural language processing, with applications including content moderation, summarization, and improvement of digital literacy.

In this project, we explore the challenge of generating spoiler content for clickbait posts. Specifically, we address two sub-tasks: identifying the most suitable spoiler type for a given teaser (phrase, passage, or multi-sentence) and generating spoiler text using the content of the linked articles.

Both tasks pose unique challenges. The first task requires semantic interpretation of vague or ambiguous headlines, and often without clear boundaries between different spoiler types. The second task demands an accurate and concise abstraction from lengthy article content, requiring the models to locate and synthesize relevant information in response to highly compressed input.

To address these issues, we adopt a modular approach. We design separate systems for spoiler classification and spoiler generation, explore multiple modeling strategies, and evaluate performance using appropriate metrics. Our final pipeline leverages pre-trained transformer architectures and task-specific input design.

Our contributions are primarily reflected in three aspects:

1. We systematically benchmarked multiple models for spoiler type classification and spoiler generation under realistic computational constraints.

2. We proposed a unified input pipeline that combines clickbait teasers with article context in a format well-suited for large language models.
3. We demonstrated strong empirical results on both sub-tasks, significantly outperforming naive baseline methods.

The report is structured as follows. Section 2 reviews related work, Section 3 introduces our methodology, Section 4 details our experiments and results, and Section 5 concludes with a summary and future directions.

2 Related Work

Clickbait detection tasks have emerged as a prominent challenge in natural language processing (NLP) due to the ubiquity of misleading or sensational online headlines. Early work by [Chakraborty et al. \(2016\)](#) explored linguistic pattern-based features and personalized classifiers, and introduced a browser extension that could reach 93% accuracy in detecting clickbait. Furthermore, [Potthast et al. \(2016\)](#) advanced the field by constructing the first publicly available clickbait dataset and presenting a strong baseline detection model, which achieved a Receiver Operating Characteristic Area Under the Curve (ROC-AUC) value of 0.79 using 215 features in a random forest classifier.

Contextualized word representations have greatly enhanced NLP performance since the development of deep learning. [Peters et al. \(2018\)](#) proposed ELMo (Embeddings from Language Models), which is pre-trained using bidirectional LSTMs to generate deep contextualized embeddings. ELMo shifted the fixed word vectors to context-aware embeddings that vary according to usage scenarios and achieved significant improvements across a wide range of NLP tasks.

Transformer-based models subsequently dominated text classification and clickbait detection tasks. [Devlin et al. \(2019\)](#) proposed BERT (Bidirectional Encoder Representations from Transformers), a masked language model that pretrains deep bidirectional representations and the first fine-tuning-based representation model to achieve outstanding results on a wide range of sentence-level and token-level tasks. In 2019, [Liu et al. \(2019\)](#) presented RoBERTa, which improved BERT by removing the next sentence prediction objective and adopting a more robust training setup. Based on BERT and RoBERTa, [He et al. \(2021\)](#) introduced

DeBERTa, which further enhances the performance through disentangled attention mechanisms and an enhanced mask decoder, successfully achieving higher training efficiency.

In 2020, [Lewis et al. \(2019\)](#) introduced BART, a denoising autoencoder that combines Bidirectional and Auto-Regressive Transformers. BART achieves generalization over BERT and GPT by pretraining on document corruption and optimizing reconstruction loss, demonstrating strong performance on text generation and comprehension tasks, including summarization, question answering, and dialogue. Together, these studies illustrate the evolutionary trajectory from feature engineering systems to contextualized and pre-trained language representations. It provides a strong foundation to support contemporary clickbait detection pipelines.

3 Approach

3.1 Task Formulation

We tackled two sub-tasks in this project. Task 1 is a multi-class classification problem that aims to predict the spoiler type, which are phrase, passage, or multi, based on a clickbait post. Task 2 is a sequence-to-sequence generation task, where the model is expected to generate spoiler content that neutralizes the ambiguity of a clickbait post using the content from the real linked article.

3.2 Input Representation and Preprocessing

For Task 1, we used only the *postText[0]* as input and the *tags[0]* field as the classification label. For Task 2, we concatenated *postText[0]*, *targetTitle*, and *targetParagraphs* into a single input string formatted as: "*post: [postText] title: [targetTitle] paragraph: [flattened targetParagraphs]*". The output target is the *spoiler* field. All data was stored in *.jsonl* format and parsed into HuggingFace datasets. Labels in Task 1 were encoded using *LabelEncoder*, and tokenization for both tasks was done using model-specific tokenizers with truncation and padding.

3.3 Model Exploration – Task 1

Our objective for Task 1 was to progressively improve clickbait classification performance by adopting models with increasing representational capacity and robustness to class imbalance. Apart from the very baseline model (predicts "passage" for all clickbait posts) provided by professor, we explored a series of models in sequence, beginning with a

simple statistical basic model and culminating in a transformer-based model with state-of-the-art contextual understanding.

TF-IDF + Logistic Regression (basic model): This classical approach was first implemented for its simplicity and interpretability and achieved F1-score of *0.453* on the test set. However, it lacked the ability to capture deep semantic relationships, limiting its effectiveness on more abstract or nuanced headlines.

bert-base-uncased: To incorporate contextual semantics, we next evaluated bert-base-uncased, which improved performance to *0.6189*. However, attempts to mitigate class imbalance using reweighting and SMOTE-style upsampling unexpectedly degraded performance to *0.208*. We supposed it was likely due to distributional shifts and overfitting on minority classes.

roberta-base: This model further improved performance, reaching a macro F1 of approximately *0.68*. We attribute this gain to RoBERTa’s robust pretraining regimen, which uses dynamic masking and a larger training corpus, enhancing its capacity for fine-grained contextual modeling.

microsoft/deberta-v3-base (final solution): This best-performing model achieved a macro F1 of *0.7317* on the test set. We fine-tuned this model using the AdamW optimizer, with a learning rate of $2e-5$, batch size of 16, and a dropout rate of 0.3 in the classification head. The architecture’s disentangled attention mechanism and explicit modeling of positional information allowed for improved generalization and semantic precision.

We observed that performance continues to improve with architecture updates and enhanced context modeling capabilities, with DeBERTa performing the best.

3.4 Model Exploration – Task 2

We proposed and evaluated four methods to generate spoilers. These methods ranged from rule-based approaches to large-scale transformer models. The final solution was selected based on empirical performance and feasibility.

Method 1- Keyword Matching (not implemented):

This approach aimed to extract salient keywords from the *postText* and match them against *targetParagraphs* to identify spoiler content. It was initially considered due to its simplicity and interpretability. However, early-stage prototyping revealed that this method suffered from poor generalization and was unable to generate abstract or paraphrased spoilers. Consequently, it was rejected in the early ideation phase.

Method 2- Fine-tuned T5 (final solution): This

approach was considered the most promising and was adopted as our final solution. We built a sequence-to-sequence generation pipeline using T5ForConditionalGeneration. During model selection, we experimented with three model sizes: t5-small, t5-base, and t5-large. Among these, T5-large delivered the best performance, achieving a test set score of *0.365*, while t5-small reached *0.252*. Training was conducted for three epochs with a learning rate of $3e-4$, using a batch size of 2 for training and 4 for evaluation, and employed standard Cross Entropy Loss. For inference, we used beam search decoding with a beam width of 4, which significantly improved output fluency compared to greedy decoding. Input and output sequences were truncated to a maximum of 512 and 64 tokens, respectively, to fit GPU memory constraints. The T5 architecture was chosen for its unified text-to-text formulation, which aligns naturally with the spoiler generation task.

Method 3- Type-Guided Generation (Abandoned):

This approach planned to guide the generation process by first classifying the spoiler type derived from Task 1. Each type would then map to a specialized generator. However, the approach was abandoned due to two major challenges: (1) the high training cost of maintaining multiple generators, and (2) blurred class boundaries between spoiler types, which led to poor label consistency and model confusion. Human annotators often disagreed on spoiler categories, further undermining the method’s viability.

Method 4: HuggingFace Summarization (not listed): We experimented with zero-shot summarization using a pre-trained summarization

model, specifically facebook/bart-large-cnn. Although the method was easy to implement, it performed poorly (test score = 0.11) and was therefore excluded from the formal evaluation. The model failed to incorporate context from *postText* and often generated generic summaries that lacked spoiler relevance, confirming its unsuitability for this task.

4 Experiments

4.1 Datasets

We used the *train.jsonl*, *val.jsonl*, and *test.jsonl* datasets provided in the competition. Each entry is explained as follows:

postText (str): The text of the clickbait post which is to be spoiled.

targetTitle (str): The title of the linked web page to classify the spoiler type (task 1) and to generate the spoiler (task 2).

targetParagraphs (List[str]): The main content of the linked web page to classify the spoiler type (task 1) and to generate the spoiler (task 2). Consists of the paragraphs of manually extracted main content.

tags (list[str]): The spoiler type ("phrase", "passage", or "multi") that is to be classified in task 1. This field is only available in the training and validation dataset.

spoiler (str): The human extracted spoiler for the clickbait post from the linked web page. This field is only available in the training and validation dataset.

During the inference phase (test phase), only the *postText*, *targetTitle*, and *targetParagraphs* fields are available. The labels (*tags*, *spoiler*) are used for evaluation and are retained.

4.2 Setup

Table 1 (see next page) outlines the experimental setup used for Task 1- clickbait classification and Task 2- spoiler generation.

For Task 1, we fine-tuned the microsoft/deberta-v3-base model using the DeBERTa tokenizer. This model is known for its disentangled attention and enhanced semantic representation, making it suitable for classification tasks. The model was trained with a batch size of 16, a learning rate of 2e-5,

and for 3 epochs using Cross Entropy Loss. Training was performed on Google Colab with an A100 GPU, and completed in approximately 1.5 minutes, reflecting the efficiency of fine-tuning on a relatively small classification task.

For Task 2, we used the t5-large model with the T5 tokenizer, which is appropriate for sequence-to-sequence tasks such as text generation. The model was trained for 3 epochs with a learning rate of 3e-4, using different batch sizes for training (2) and evaluation (4) due to memory constraints. Similar to Task 1, we used Cross Entropy Loss as the objective function. However, due to the increased complexity and size of the model, as well as the nature of the generation task, training took significantly longer of approximately 30 minutes on the same A100 GPU.

4.3 Evaluation Metrics

We evaluated the performance of our models using task-specific metrics that align with the respective goals of classification and generation.

4.3.1 Task 1: Macro-Averaged F1-Score

For the multi-class classification task of identifying clickbait categories, we employed the macro-averaged F1-score, which is particularly appropriate for imbalanced datasets. The F1-score is the harmonic mean of precision and recall, defined as:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Where:

- $\text{Precision} = \frac{TP}{TP+FP}$: Proportion of true positives among predicted positives.
- $\text{Recall} = \frac{TP}{TP+FN}$: Proportion of true positives among actual positives.

The macro-averaged F1-score is then computed as:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c$$

Where C is the total number of classes.

4.3.2 Task 2: METEOR Score

For the spoiler generation task, we adopted the METEOR score (Metric for Evaluation of Translation with Explicit Ordering), a reference-based metric that aligns generated text with human-written reference answers at the semantic level. The METEOR score is calculated as:

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{Penalty})$$

Parameter	Task 1	Task 2
Model	deberta-v3-base	t5-large
Tokenizer	DeBERTa tokenizer	T5 tokenizer
Learning Rate	2e-5	3e-4
Batch Size	16	2(train) / 4(eval)
Epochs	3	3
Loss	Cross Entropy Loss	Cross Entropy Loss
Environment	Google Colab (A100 GPU)	Google Colab (A100 GPU)
Training Time	1.5 minutes	30 minutes

Table 1: Experimental Setup for Task 1 (Classification) and Task 2 (Generation)

Task	Baseline Strategy	Metric	Baseline Score	Final Score	Relative Gain
1	Predict all as "passage"	Macro-F1	0.2085	0.7317	+250%
2	Use <i>targetTitle</i> as spoiler	METEOR	0.0924	0.3650	+295%

Table 2: Performance Comparison Between Baseline Strategies and Final Models for Task 1 and Task 2

Where:

- $F_{\text{mean}} = \frac{10 \cdot P \cdot R}{R + 9P}$: A weighted harmonic mean of unigram precision P and recall R .
- $\text{Penalty} = \gamma \cdot (\frac{ch}{m})^\theta$: Penalty for fragmented matches, where ch is the number of chunks, and m is the number of matched unigrams.

Typical values are: $\gamma = 0.5$, $\theta = 3$

4.4 Results

Table 2 summarizes the evaluation results comparing the baseline strategies to the final model performances for both tasks. These results indicate that pre-trained transformers and model selection are both effective for classification and generation tasks. Notably, task 1 benefits from deeper contextual encoders, while task 2 achieves significant improvements with sequence-level fine-tuning.

4.4.1 microsoft / deberta-v3-base model

For Task 1, the final model utilized microsoft / deberta-v3-base, enhanced with a dropout layer ($p = 0.3$), ReLU activation, and a linear classification head. This architecture achieved the best performance on the test set with a macro F1 of 0.7317 . DeBERTa’s disentangled attention and rich contextual representations demonstrate effectiveness in capturing the subtle linguistic clues present in different types of clickbait. The additional dropout layer and activation layer, respectively, improved regularization and nonlinear feature extraction capabilities. Overall, the model performs well in this task-specific context, but classification errors still

exist in some cases where the headline semantics are ambiguous.

The outstanding performance of microsoft / deberta-v3-base can be attributed to its core architectural innovations. Unlike traditional BERT models, which simply aggregate token embeddings and position embeddings, DeBERTa introduces disentangled attention that encodes the content and position of tokens separately, and computes attention scores using both. This enables the model to more accurately capture fine-grained contextual signals in ambiguous or multi-label headlines. Furthermore, the v3 version improves pretraining efficiency through an enhanced masked decoder, which leads to stronger representations for downstream classification. These design improvements, combined with light task-specific modifications (dropout + ReLU + linear head), result in substantial improvement over the naive baseline.

4.4.2 T5-large model

For Task 2, the final solution was based on a fine-tuned T5-large model, which achieved the highest METEOR score of 0.365 among all evaluated variants (t5-small: 0.252 , t5-base: 0.317). The performance improvement of T5-large can be attributed to its encoder-decoder architecture, which effectively captures long-range dependencies in the passage and headline. Additionally, the span-corruption pretraining objective of T5 enhances its ability to infer missing context, enabling the generation of spoiler content that is both semantically consistent and coherent. Beam search decoding, as opposed to greedy decoding, further improved the

quality of generated outputs by balancing lexical diversity and relevance. While the model achieved substantial improvements over the baseline, some outputs remained overly generic, suggesting opportunities for future enhancements with reranking or factual consistency mechanisms.

5 Conclusion

In this study, we addressed the problem of identifying clickbait spoilers through two complementary subtasks: spoiler type classification and spoiler text generation. Our approach combines modular modeling with incremental refinement, evolving from traditional baseline models to state-of-the-art transformer architectures.

In the classification task, we benchmarked multiple models and observed that performance significantly improved when transitioning from TF-IDF and early transformer models to further advanced architectures. The final classifier built based on deberta-v3-base achieved an impressive macro F1 score of 0.7317.

In the generation task, we experimented with various strategies and ultimately deployed a fine-tuned t5-large model in a sequence-to-sequence setting. This setting achieved a METEOR score of 0.365, significantly outperforming naive baselines and off-the-shelf summarization, by combining structured inputs of teaser text and article content.

Our findings confirm the value of pre-trained language models in both classification and generation pipelines, especially when adapted to specific tasks. Future work may include exploring unified models that can perform classification and generation simultaneously, using external commonsense knowledge, and building controllable generation mechanisms that can dynamically adjust style and length. Overall, we expect our framework can contribute to more transparent, informative, and user-friendly content in the digital media world.

References

Abhijnan Chakraborty, Bhargavi Paranjape, Sourya Kakarla, and Niloy Ganguly. 2016. [Stop clickbait: Detecting and preventing clickbaits in online news media](#). *Preprint*, arXiv:1610.09786.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [Bert: Pre-training of deep bidirectional transformers for language understanding](#). *Preprint*, arXiv:1810.04805.

Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. [Deberta: Decoding-enhanced bert with disentangled attention](#). *Preprint*, arXiv:2006.03654.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. [Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension](#). *Preprint*, arXiv:1910.13461.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. [Roberta: A robustly optimized bert pretraining approach](#). *Preprint*, arXiv:1907.11692.

Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. [Deep contextualized word representations](#). *Preprint*, arXiv:1802.05365.

Martin Potthast, Sebastian Köpsel, Benno Stein, and Matthias Hagen. 2016. Clickbait detection. In *Advances in Information Retrieval*, pages 810–817, Cham. Springer International Publishing.